Machine Learning

The Art and Science of Algorithms that Make Sense of Data

Dr. Mitra Baratchi

20 March 2026

Leiden Institute of Advanced Computer Science - Leiden University

Session 7: Distance-based models [1]

Reading material

→ Chapter 8, Machine learning: the art and science of algorithms that make sense of data

[1] The slides partially include material based on the slides accompanying the book: ***Machine learning: the art and science of algorithms that make sense of data*** Cambridge University Press, 2012.

Table of Contents

Distance-based models

Distance based algorithms

Distance-based algorithms are machine learning algorithms that classify instances by computing distances between these instances and a number of internally stored exemplars.

Distance based algorithms

Distance-based algorithms are machine learning algorithms that classify instances by computing distances between these instances and a number of internally stored exemplars.

- **Exemplars** that are closest to the instance have the largest influence on the classification assigned to the instance.
- **Neighbours** can be near exemplars.

Distance based algorithms

How to calculate distance?

Minkowski distance i

Minkowski distance

If $\mathscr{X} = \mathbb{R}^d$, the *Minkowski distance* or *p-norm* of order $p > 0$ is

$$\mathrm{Dis}_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{j=1}^{d} |x_j - y_j|^p \right)^{1/p} = ||\mathbf{x} - \mathbf{y}||_p$$

Hamming and edit distance

Hamming distance

0-norm counts the number of non-zero elements in a vector. The corresponding distance then counts the number of positions in which vectors $\mathbf{x}$ and $\mathbf{y}$ differ:

$$\mathrm{Dis}_0(\mathbf{x},\mathbf{y}) = \sum_{j=1}^{d} (x_j - y_j)^0 = \sum_{j=1}^{d} I[x_j = y_j]$$

If $\mathbf{x}$ and $\mathbf{y}$ are binary strings, this is also called the *Hamming distance*. (the number of bits that need to be flipped to change). For non-binary strings this can be generalised to the notion of *edit distance* or *Levenshtein distance*.

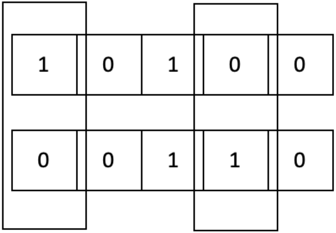
Hamming distance

1	0	1	0	0

0	0	1	1	0

Hamming distance equal to ?

Hamming distance



Hamming distance is equal to 2

Euclidean versus Manhattan distance

Manhattan distance

The 1-*norm* denotes *Manhattan distance*, also called *city-block distance*:

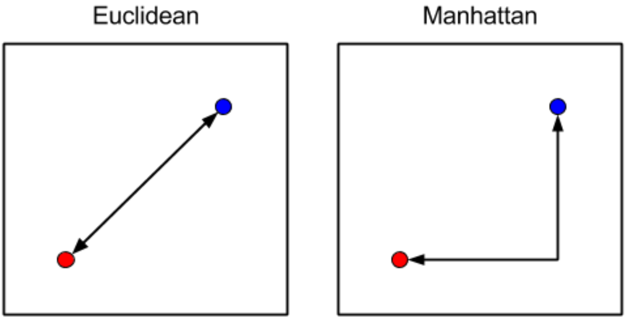
$$\mathrm{Dis}_1(\mathbf{x}, \mathbf{y}) = \sum_{j=1}^{d} |x_j - y_j|$$

Euclidean distance

2-*norm* refers to the familiar *Euclidean distance*

$$\mathrm{Dis}_2(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^{d} (x_j - y_j)^2}$$

Euclidean versus Manhattan distance



(left) *Euclidan distance* ***(right)*** *Manhattan distance* [2]

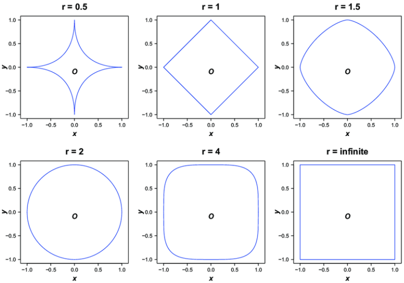
[2] imagesourcehttps://subscription.packtpub.com/book/big_data_and_business_intelligence/9781785882104/6/ch06lvl1sec40/measuring-distanceorsimilarity

Chebyshev distance

Chebyshev distance

Chebyshev distance: if we let p grow larger, the distance will be more and more dominated by the largest coordinate-wise distance, from which we can infer that $\mathrm{Dis}_{\infty}(\mathbf{x}, \mathbf{y}) = \max_j |x_j - y_j|$.

Chebyshev distance



Chebyshev distance for different values of $p = r$ [3]

All points lie on a similar distance from the origin

[3] imagesourcehttps://www.researchgate.net/figure/Points-in-a-2-dimensional-space-at-a-distance-of-1-from-the-center-O-using-the_fig1_283088044

Distance metric

Are these distance metrics good metrics? Can we define our own distance metric?

Distance metric

Distance metric

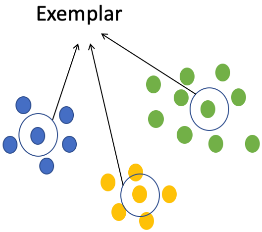
Given an instance space $\mathscr{X}$, a *distance metric* is a function $\mathrm{Dis} : \mathscr{X} \times \mathscr{X} \to \mathbb{R}$ such that for any $x, y, z \in \mathscr{X}$:

- Distances between a point and itself are zero: $\mathrm{Dis}(x, x) = 0$;
- All other distances are larger than zero: if $x \neq y$ then $\mathrm{Dis}(x, y) > 0$;
- Distances are symmetric: $\mathrm{Dis}(y, x) = \mathrm{Dis}(x, y)$;
- Triangle inequality: detours can not shorten the distance: $\mathrm{Dis}(x, z) \leq \mathrm{Dis}(x, y) + \mathrm{Dis}(y, z)$.

If the second condition is weakened to a non-strict inequality – i.e., $\mathrm{Dis}(x, y)$ may be zero even if $x \neq y$ – the function Dis is called a *pseudo-metric*.

Neighbours and exemplars

Exemplars and neighbors



Exemplars are prototypical instances within clusters/classes

- How to define exemplars?
- How to define decision rules based on nearest neighbors/exemplars?

Exemplars

Can you think of a good exemplar?

Mean and median as exemplars

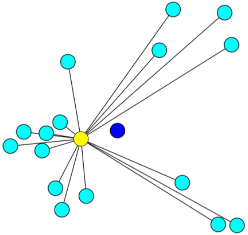
- Geometric mean (centroid)
- Geometric median (medoid)

Mean and median as exemplars

- Geometric mean (centroid)
- Geometric median (medoid)

→ Which one is better?

Centroid and medoid



A centroid (blue point) and medoid (yellow point) [4]

The blue point is the centroid (does not happen in data)

The yellow point is the medoid (happens in data) → more time consuming to calculate

[4] image source: https://en.wikipedia.org/wiki/Geometric_median

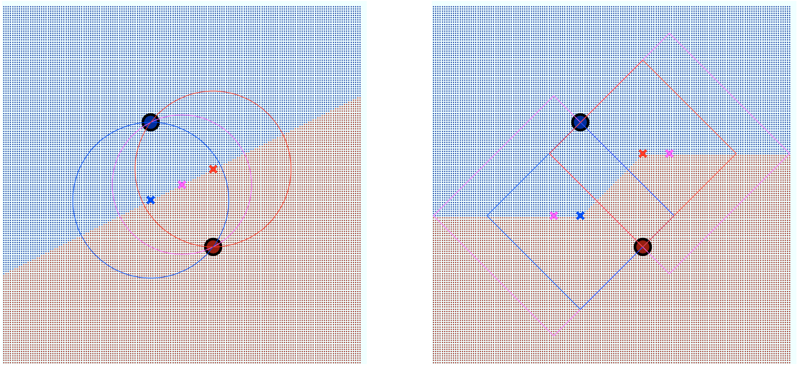
A basic distance-based classifier

Creating a distance-based classifier.

- Option: using the decision rule that classifies an instance to the class of the *nearest* exemplar

What would a decision boundary look like with distance-based perspective (e.g., different distance measures)?

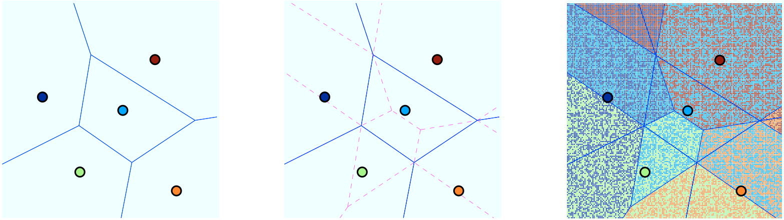
Two-exemplar decision boundaries



(left) *For two exemplars, the nearest-exemplar decision rule with Euclidean distance results in a linear decision boundary coinciding with the perpendicular bisector of the line connecting the two exemplars.* ***(right)*** Using Manhattan distance, the circles are replaced by diamonds.

→ the 2-norm decision boundary is more regular

One vs two nearest neighbours



(left) Voronoi tesselation for five exemplars. **(middle)** Taking the two nearest exemplars into account leads to a further subdivision of each Voronoi cell. **(right)** The shading indicates which exemplars contribute to which cell.

→ Since the number of classes is typically much lower than the number of exemplars, decision rules often take more than one nearest exemplar into account (or k-nearest exemplar).

Distance-based models

To summarise, the main ingredients of distance-based models are

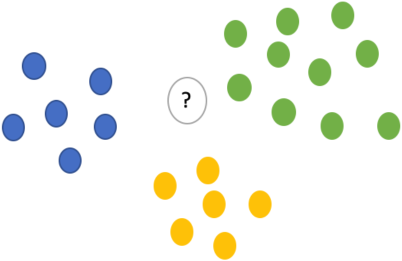
- **Distance metrics:** Euclidean, Manhattan, or Minkowski, among many others;
- **Exemplars:** centroids that find a centre of mass according to a chosen distance metric, or medoids that find the most centrally located data point;
- **Distance-based decision rules:** which take a vote among the k-nearest exemplars.

Table of content

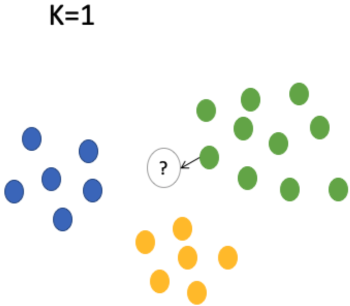
Nearest-neighbour classifier

Instead of finding the mean of instances as an exemplar we will take each instance as one.

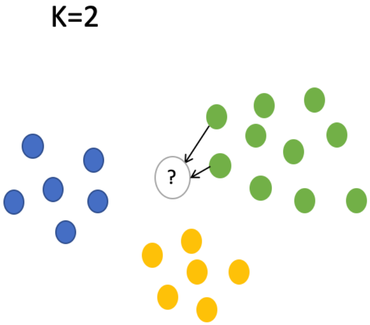
K-nearest neighbour (KNN)



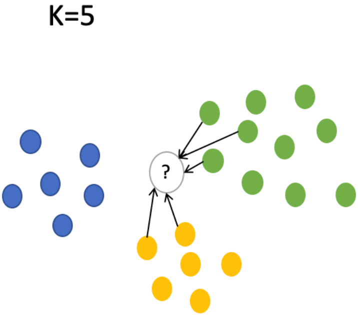
K-nearest neighbour



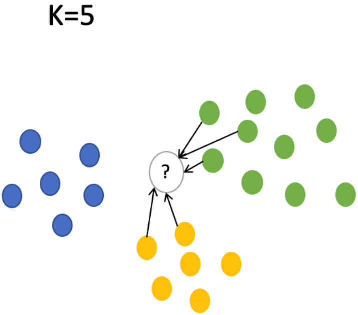
K-nearest neighbour



K-nearest neighbour



K-nearest neighbour



The class with the majority vote is selected.

Extending KNN to regression

- Easily adapted to real-valued targets. We can take the mean value of the exemplars (or weight them based on their distance)

Curse of dimensionality

How does the **dimensionality** of the data impact the distance-based approach?

Curse of dimensionality

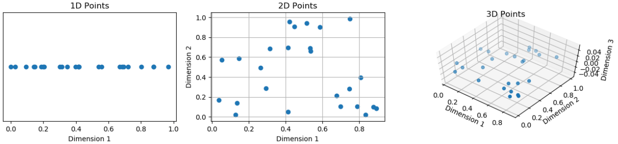
How does the **dimensionality** of the data impact the distance-based approach?

What is meant by dimension? → the number of features

Curse of dimensionality

https://colab.research.google.com/drive/1y4CsrzvmqZTbHglNpIWLhkKSpas4PqzS?usp=sharing

Curse of dimensionality



Random points in different dimensions

Curse of dimensionality

In high-dimensional spaces everything is far away from everything and so pairwise distances are uninformative

- In reality a lot of data sets we receive are high dimensional (e.g., images, time-series)

Nearest-neighbour classifier properties

- Nearly perfect separation of classes on the training set
- Easily adapted to real-valued targets and structured objects
- Unbalanced complexity
  - Training is storing n exemplars → $O(n)$
  - Prediction of each instance involves comparisons with n exemplars → $O(n)$
- Perfectly separates training data → low bias but high variance
  - By increasing the number of neighbours k we would increase bias and decrease variance

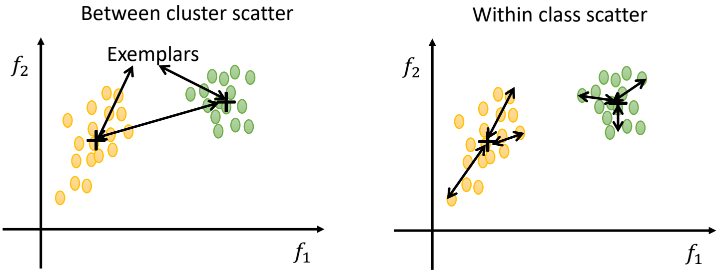
Distance-based predictive clustering

Distance-based clustering

- **Predictive clustering:** use a distance metric, a way to construct exemplars and a distance-based decision rule to create clusters that are compact with respect to the distance metric.
- **Descriptive clustering:** trees called dendrograms purely defined in terms of a distance measure are used. They partition (and describe) the given data rather than the entire instance space.

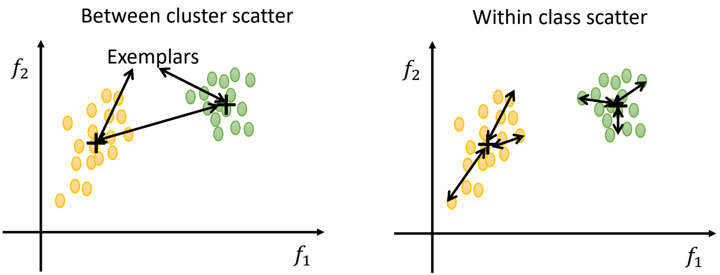
Table of content

Predictive clustering



(left) *Between-cluster scatter* ***(right)*** Within-cluster scatter

Predictive clustering



(left) *Between-cluster scatter* ***(right)*** Within-cluster scatter

The goal is to find compact clusters → minimise within-cluster scatter

K-means clustering

The goal is to minimise the total scatter over all clusters.

K-means clustering problem

The *K-means clustering problem* is to find a partition that minimises the total **within-cluster scatter**.

Scatter of $\mathbf{X}$ is defined as $\mathrm{Scat}(\mathbf{X}) = \sum_{i=1}^{n} ||\mathbf{X}_{i\cdot} - \boldsymbol{\mu}||^2$

K-means clustering

Algorithm KMeans(D, K) – K-means clustering using Euclidean distance Dis_2.

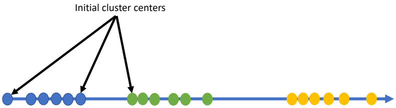
In : data $D \subseteq \mathbb{R}^d$; number of clusters $K \in \mathbb{N}$.
O : K cluster means $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K \in \mathbb{R}^d$.

randomly initialise K vectors $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K \in \mathbb{R}^d$;
repeat
  assign each $\mathbf{x} \in D$ to $\arg\min_j \mathrm{Dis}_2(\mathbf{x}, \boldsymbol{\mu}_j)$;
  for $j = 1$ to K **do**
    $D_j \leftarrow \{\mathbf{x} \in D | \mathbf{x}$ assigned to cluster $j\}$;
    $\boldsymbol{\mu}_j = \frac{1}{|D_j|} \sum_{\mathbf{x} \in D_j} \mathbf{x}$;
  end
until no change in $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$;
return $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$;

K-means clustering

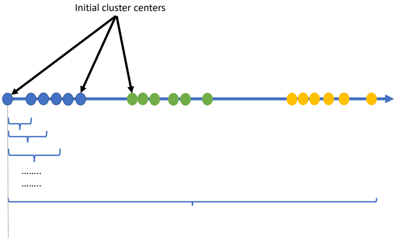
A number of points from different groups

K-means clustering



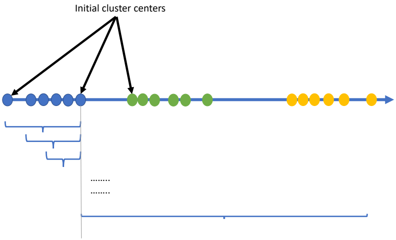
The initial randomly assigned cluster centroids.

K-means clustering



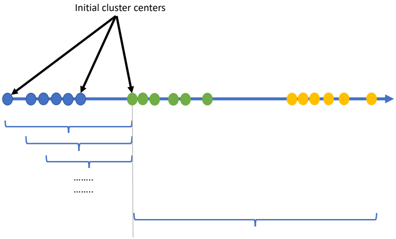
Calculating the distance of each point from the cluster centroids.

K-means clustering



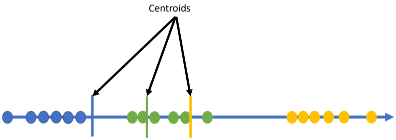
Calculating the distance of each point from the cluster centroids.

K-means clustering



Calculating the distance of each point from the cluster centroids.

K-means clustering



Re-calculating the centroids.

Weak points → The algorithm can be impacted by the starting points, you also need to know the number of cluster in advance

Table of content

Centroid and medoid

Medoid

Finding a medoid requires us to calculate, for each data point, the total distance to all other data points, in order to choose the point that minimises it. Regardless of the distance metric used, this is an $O(n^2)$ operation for n points.

K-medoids clustering

Algorithm KMedoids(D, K, Dis) – K-medoids clustering using arbitrary distance metric Dis.

In : data $D \subseteq \mathscr{X}$; number of clusters $K \in \mathbb{N}$;
distance metric $\mathrm{Dis} : \mathscr{X} \times \mathscr{X} \to \mathbb{R}$.
O : K medoids $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K \in D$, representing a predictive clustering of $\mathscr{X}$.

```
randomly pick K data points μ_1, ..., μ_K ∈ D;
repeat
    assign each x ∈ D to argmin_j Dis(x, μ_j);
    for j = 1 to k do
        D_j ← {x ∈ D | x assigned to cluster j};
        μ_j = argmin_{x ∈ D_j} Σ_{x' ∈ D_j} Dis(x, x');
    end
until no change in μ_1, ..., μ_K;
return μ_1, ..., μ_K;
```

Table of content

Inertia

Inertia

The K-means algorithm aims to choose centroids that minimise the *inertia* or *within-cluster scatter*:

$$\sum_{i}^{n} min_{\mu_i \in C} ||x_i - \mu_i||^2$$

→ Inertia shows how internally coherent and compact clusters are

Silhouettes

Silhouettes

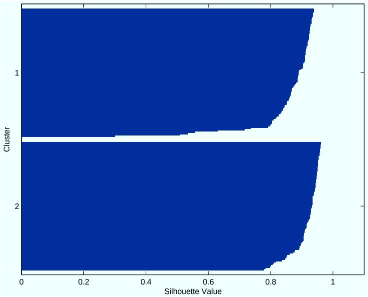
A *silhouette* then sorts and plots $s(\mathbf{x})$ for each instance, grouped by cluster.

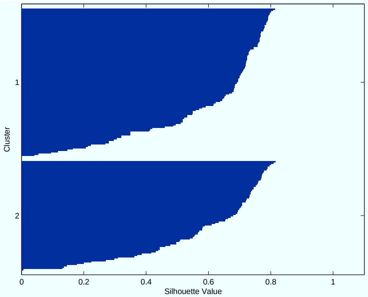
$$s(\mathbf{x}_i) = \frac{b(\mathbf{x}_i) - a(\mathbf{x}_i)}{\max(a(\mathbf{x}_i), b(\mathbf{x}_i))}$$

- $a(\mathbf{x}_i) = d(\mathbf{x}_i, D_{j(i)})$ is the average distance of $\mathbf{x}_i$ to the points in its own cluster $D_{j(i)}$
- $b(\mathbf{x}_i) = \min_{k \neq j(i)} d(\mathbf{x}_i, D_k)$ is the average distance to the points in its neighbouring cluster
- $d(\mathbf{x}_i, D_j)$ denotes the average distance of $\mathbf{x}_i$ to the data points in cluster D_j (where $\mathbf{x}_i$ belongs to)

→ We want a high value for b and a low value for a

Silhouettes





(left) *Silhouette for the clustering using squared Euclidean distance. Almost all points have a high $s(\mathbf{x})$, which means that they are much closer, on average, to the other members of their cluster than to the members of the neighbouring cluster.* ***(right)*** The silhouette for the clustering is much less convincing.

Descriptive hierarchical clustering

Descriptive clustering

An example of descriptive clustering ...

Hierarchical cluster

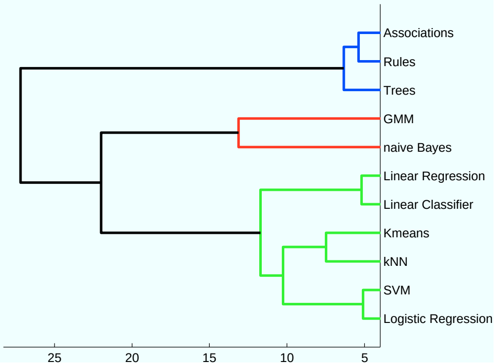
Hierarchical clustering

The K-means algorithms is **flat**. Hierarchical clustering provides a taxonomy of instances within the cluster, or where clusters can be merged with each other.

Hierarchical clustering

We typically use graphs called *dendograms* to present the hierarchical clustering results.

Hierarchical clustering example



A dendrogram (printed left to right to improve readability) constructed by hierarchical clustering

Dendrogram

Dendogram

Given a data set D, a *dendrogram* is a binary tree with the elements of D at its leaves. An internal node of the tree represents the subset of elements in the leaves of the subtree rooted at that node. The level of a node is the distance between the two clusters represented by the children of the node. Leaves have level 0.

Linkage function

In order to create dendograms we need a way to compare clusters:

Linkage function

In order to create dendograms we need a way to compare clusters:

Linkage function

A *linkage function* $L : 2^{\mathscr{X}} \times 2^{\mathscr{X}} \to \mathbb{R}$ calculates the distance between arbitrary subsets of the instance space, given a distance metric $\mathrm{Dis} : \mathscr{X} \times \mathscr{X} \to \mathbb{R}$.

Linkage function

The most common linkage functions are as follows:

Single linkage the *smallest* pairwise distance between elements from each cluster:

$$L_{\text{single}}(A,B) = \min_{x\in A, y\in B} \mathrm{Dis}(x,y)$$

Complete linkage the *largest* point-wise distance:

$$L_{\text{complete}}(A,B) = \max_{x\in A, y\in B} \mathrm{Dis}(x,y)$$

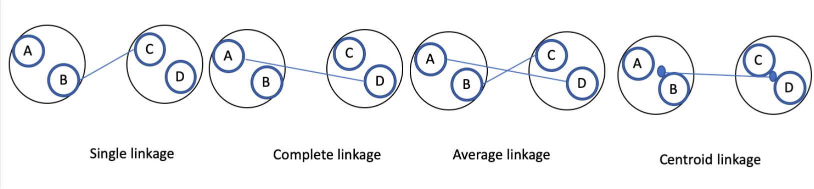
Average linkage *average* point-wise distance:

$$L_{\text{average}}(A,B) = \frac{\sum_{x\in A, y\in B} \mathrm{Dis}(x,y)}{|A|\cdot|B|}$$

Centroid linkage the point distance between the cluster means:

$$L_{\text{centroid}}(A,B) = \mathrm{Dis}\left(\frac{\sum_{x\in A} x}{|A|}, \frac{\sum_{y\in B} y}{|B|}\right)$$

Linkage functions



The different kinds of linkage

Clearly, all these linkage functions coincide for singleton clusters: $L(\{x\},\{y\}) = \mathrm{Dis}(x,y)$. However, for larger clusters they start to diverge.

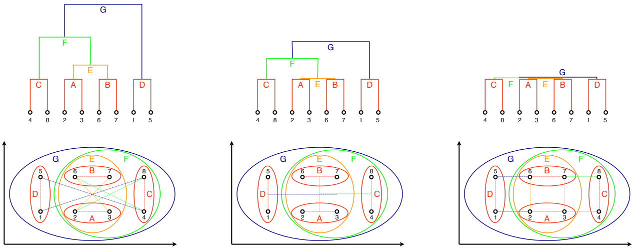
Hierarchical agglomerative clustering

Algorithm HAC(D, L) – Hierarchical agglomerative clustering.

In : data $D \subseteq \mathscr{X}$; linkage function $L : 2^{\mathscr{X}} \times 2^{\mathscr{X}} \to \mathbb{R}$ defined in terms of distance metric.
O : a dendrogram representing a descriptive clustering of D.

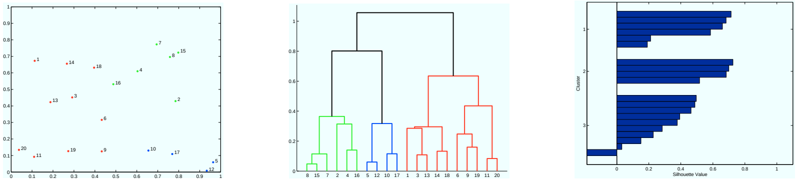
initialise clusters to singleton data points;
create a leaf at level 0 for every singleton cluster;
repeat
  find the pair of clusters X, Y with lowest linkage l, and merge;
  create a parent of X, Y at level l;
until all data points are in one cluster;
return the constructed binary tree with linkage levels;

Linkage matters



(left) Complete linkage defines cluster distance as the largest pairwise distance between elements from each cluster, indicated by the coloured lines between data points. **(middle)** Centroid linkage defines the distance between clusters as the distance between their means. Notice that E obtains the same linkage as A and B, and so the latter clusters effectively disappear. **(right)** Single linkage defines the distance between clusters as the smallest pairwise distance. The dendrogram all but collapses, which means that no meaningful clusters are found in the given grid configuration.

A spurious clustering



(left) 20 data points, generated by uniform random sampling. **(middle)** The dendrogram generated from complete linkage. The three clusters suggested by the dendrogram are spurious as they cannot be observed in the data. **(right)** The rapidly decreasing silhouette values in each cluster confirm the absence of a strong cluster structure. Point 18 has a negative silhouette value as it is on average closer to the green points than to the other red points.

Lessons learned

- Different distance metrics based on Minkowski distance (e.g., Euclidean distance)
- Neighbours and exemplars (e.g., Mean and Median)
- K-Nearest neighbour classifier: low bias and high variance, it can be applied to real-valued targets
- K-means clustering: predictive distance based clustering clustering
- Hierarchical clustering: descriptive distance based clustering